Binary Search Tree (BST)

Introduction to Binary Search Tree (BST)

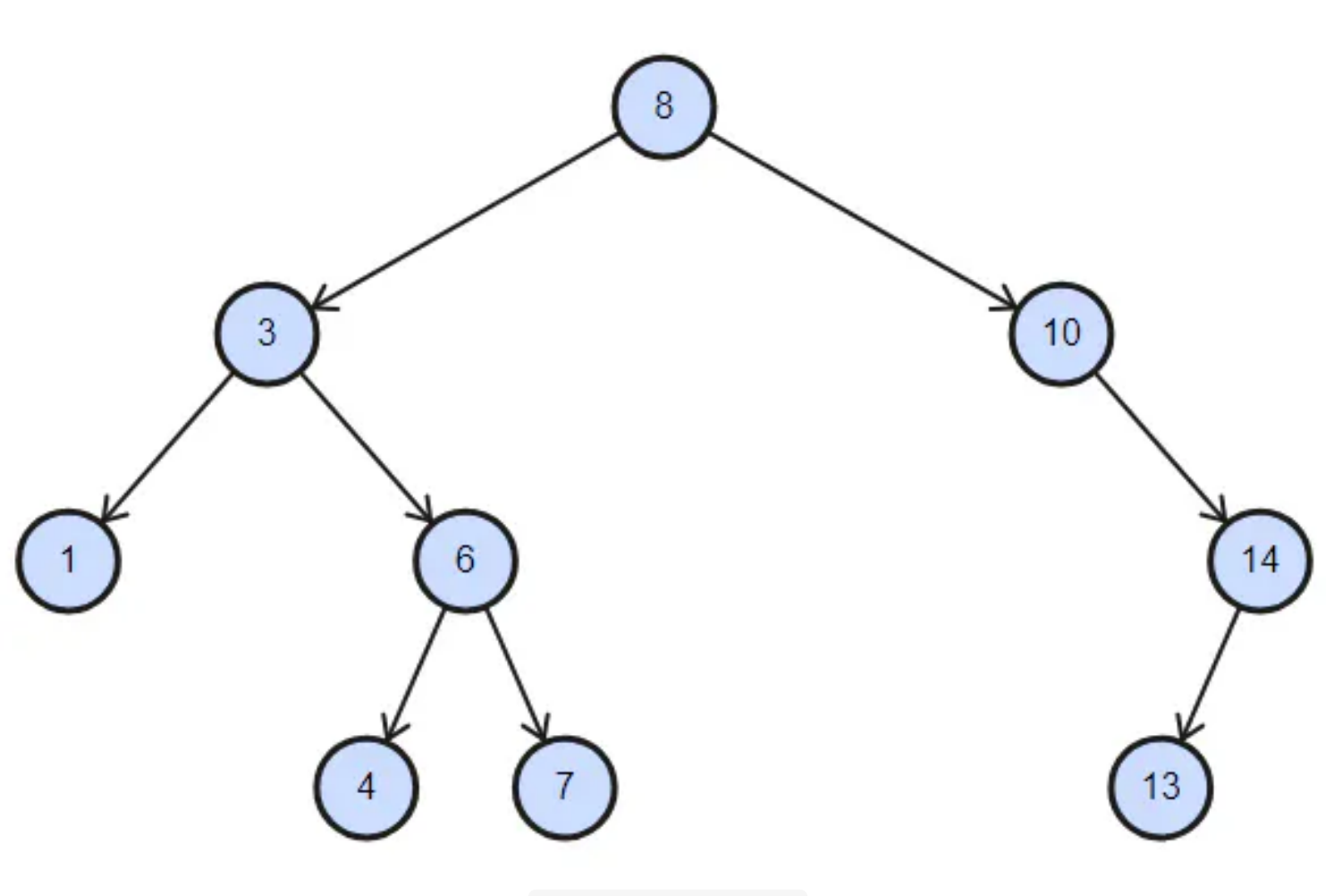
A Binary Search Tree (BST) is a fundamental data structure that organizes elements in a hierarchical manner, allowing for efficient searching, insertion, deletion, and traversal operations. In this section, we will explore the characteristics of a BST, its benefits, applications, ADT operations, time and space complexity, as well as potential issues and solutions.

Definition and Characteristics of BST

A Binary Search Tree is a binary tree in which each node has at most two children, referred to as the left child and the right child. The BST follows a specific property: for any given node, all nodes in its left subtree have values less than or equal to the node's value, and all nodes in its right subtree have values greater than the node's value.

Example:

Consider the following BST:



Binary Search Tree

In this example, the tree satisfies the BST property since, for each node, all nodes in its left subtree have values less than the node's value, and all nodes in its right subtree have values greater than the node's value.

Benefits and Applications of BST

BSTs provide efficient searching operations due to their hierarchical structure and the BST property. The search can be performed in O(log n) time in a balanced BST.

1. **Efficient Searching**: BSTs provide fast searching capabilities with an average time complexity of O(log n) for balanced trees. This makes them suitable for applications that require quick data retrieval based on a key, such as database indexing and dictionary implementations.
2. **Ordered Data Storage**: The inherent order of BSTs makes them suitable for scenarios where data needs to be stored in a sorted manner. For example, BSTs can maintain sorted lists or retrieve elements in a particular order.
3. **Binary Search Tree in Databases**: BSTs are used in databases to create indexes that allow for faster data retrieval based on specific fields. Using BSTs for indexing, databases can efficiently handle large datasets and complex queries.
4. **Binary Search Tree in File Systems**: File systems often use BSTs to organize and manage directory structures. The hierarchical nature of BSTs aligns well with the hierarchical structure of directories, enabling efficient file retrieval.

BST Abstract Data Type (ADT)

The BST ADT covers various operations to manipulate and manage the tree efficiently. The basic operations typically supported by a BST ADT include the following:

1. Insertion
2. Update
3. Deletion
4. Search
5. In-order Traversal
6. Pre-order Traversal
7. Post-order Traversal

Let's start by discussing our data model. We will use a Node class to store a single node of BST. Here is what our Node class looks like:

Java

```java
class Node<T> {
    public T data;
    public Node<T> left;
    public Node<T> right;

    public Node(T value) {
        data = value;
        left = null;
        right = null;
    }
}
```

The above class represents a single node of a binary tree. It has the following members:

- `info` : A template member that stores the value of the node.
- `left` : A pointer to the left child node.
- `right` : A pointer to the right child node.

The class has two constructors:

- `Node()` : Default constructor that initializes `left` and `right` pointers to null (0).
- `Node(int v)` : Constructor that takes a value `v` and initializes `info` with it. It also sets `left` and `right` pointers to null.

ADT Class

Now, let's look at the BST ADT class definition.

Java

```java
public class BST<T extends Comparable<T>> {
    private Node<T> root;

    private void inOrderHelper(Node<T> p);
    private void preOrderHelper(Node<T> p);
    private void postOrderHelper(Node<T> p);
    private Node<T> findMin(Node<T> node);
    private Node<T> deleteNode(Node<T> root, T node);

    public BST();
    public void insert(T node);
    public void deleteMethod(T node);
    public boolean search(T node);
    public void inOrder();
    public void preOrder();
    public void postOrder();
}
```

The above code defines a binary search tree class. It uses templates to support different data types for the elements stored in the tree.

The class contains private helper functions for in-order, pre-order, and post-order traversals, as well as for finding the minimum node in a subtree, deleting a node, and updating a node's value.

The public interface provides functions to insert, delete, search, and update nodes in the BST. Additionally, it offers functions to perform in-order, pre-order, and post-order traversals of the tree. The code is designed to facilitate managing and manipulating binary search trees with various data types.

Let's discuss each ADT operation of BST:

Insertion:

The insertion operation adds a new element to the BST while maintaining the BST property. The BST property ensures that the left subtree contains nodes with values less than or equal to the current node's value, and the right subtree contains nodes with values greater than the current node's value.

Here is the implementation of the insert method:

Java

```java
public void insert(T value) {
    Node<T> newNode = new Node<>(value);

    if (root == null) {
        root = newNode;
    } else {
        Node<T> current = root;
        Node<T> parent = null;

        while (current != null) {
            parent = current;
            if (value.compareTo(current.data) < 0) {
                current = current.left;
            } else {
                current = current.right;
            }
        }

        if (value.compareTo(parent.data) < 0) {
            parent.left = newNode;
        } else {
```

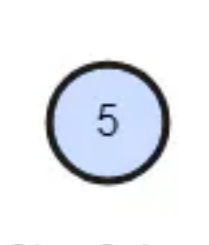
The given code is a template function to insert a new node with a value `node` into a binary search tree (BST) while maintaining the BST property. If the tree is empty, the new node becomes the root. Otherwise, the function traverses the tree to find the appropriate position for insertion.

It compares the value of the `node` with each node along the path and goes left if the value is less and right if the value is greater. Based on the comparison, when the correct position is found, the new node becomes the left or right child of the parent node. The function efficiently inserts the node into the BST while keeping the tree ordered.

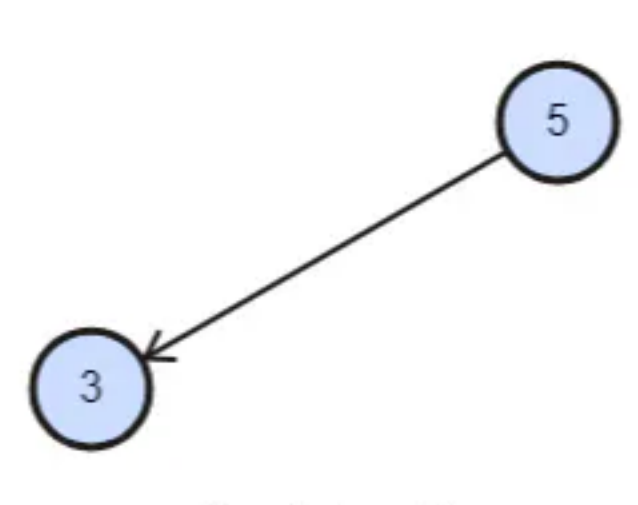
Example:

Consider an empty BST. We insert the values 5, 3, 7, 2, 4, 6, and 8 in the following order:

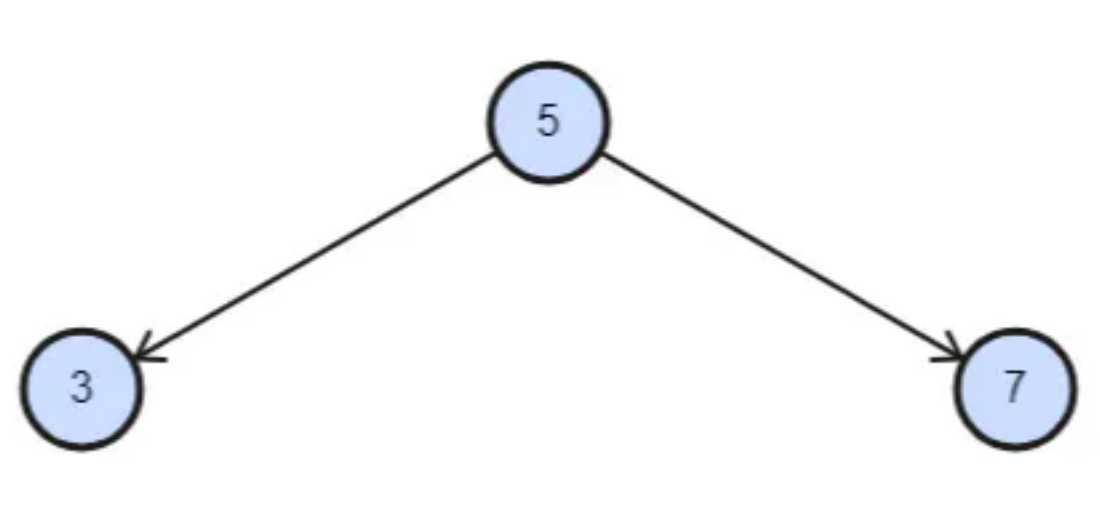
Step 1: Insert 5



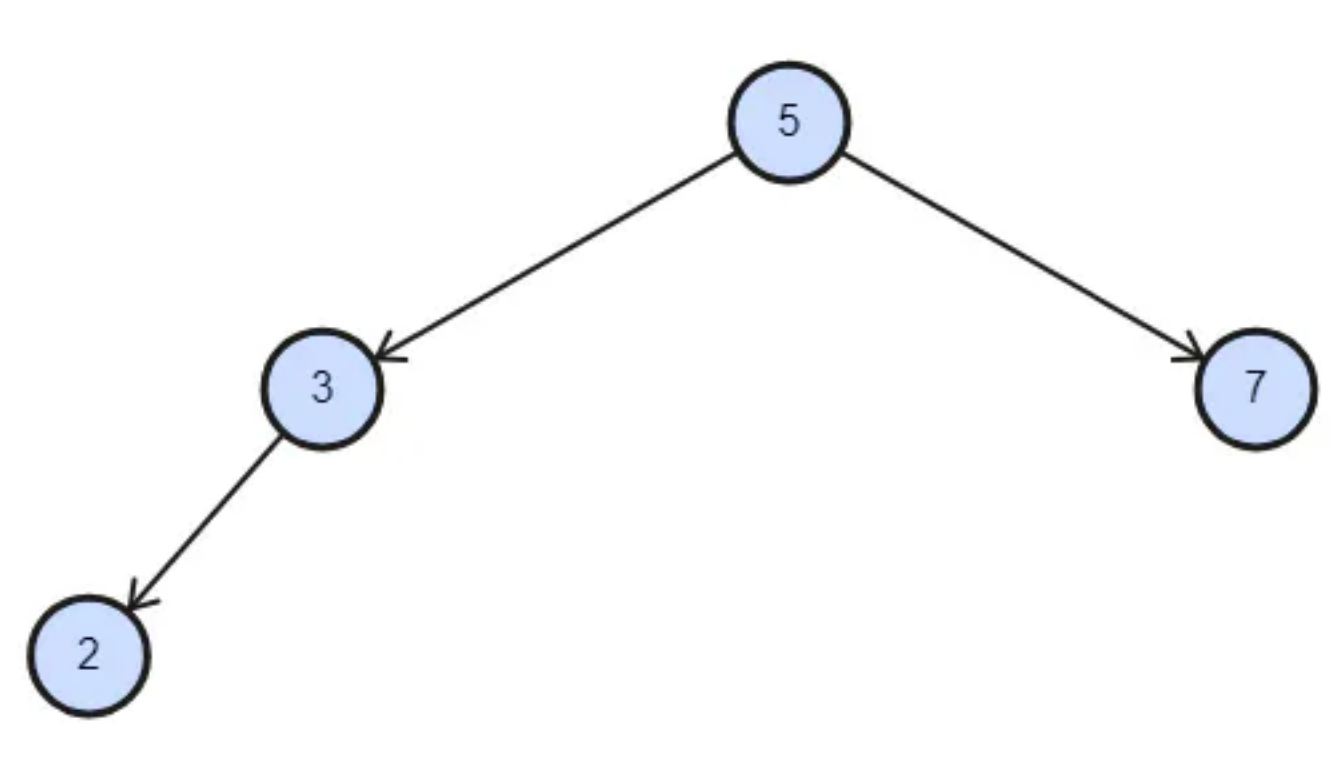
Step 2: Insert 3



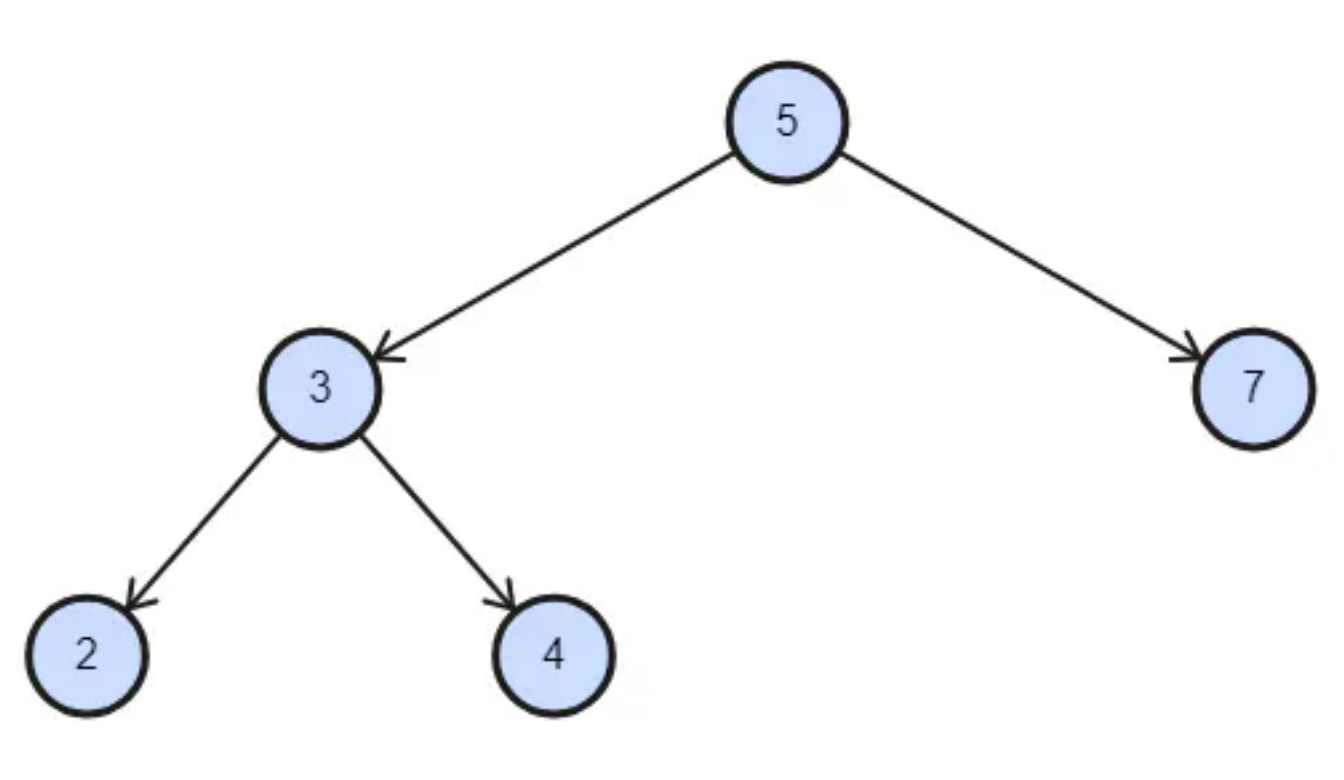
Step 3: Insert 7



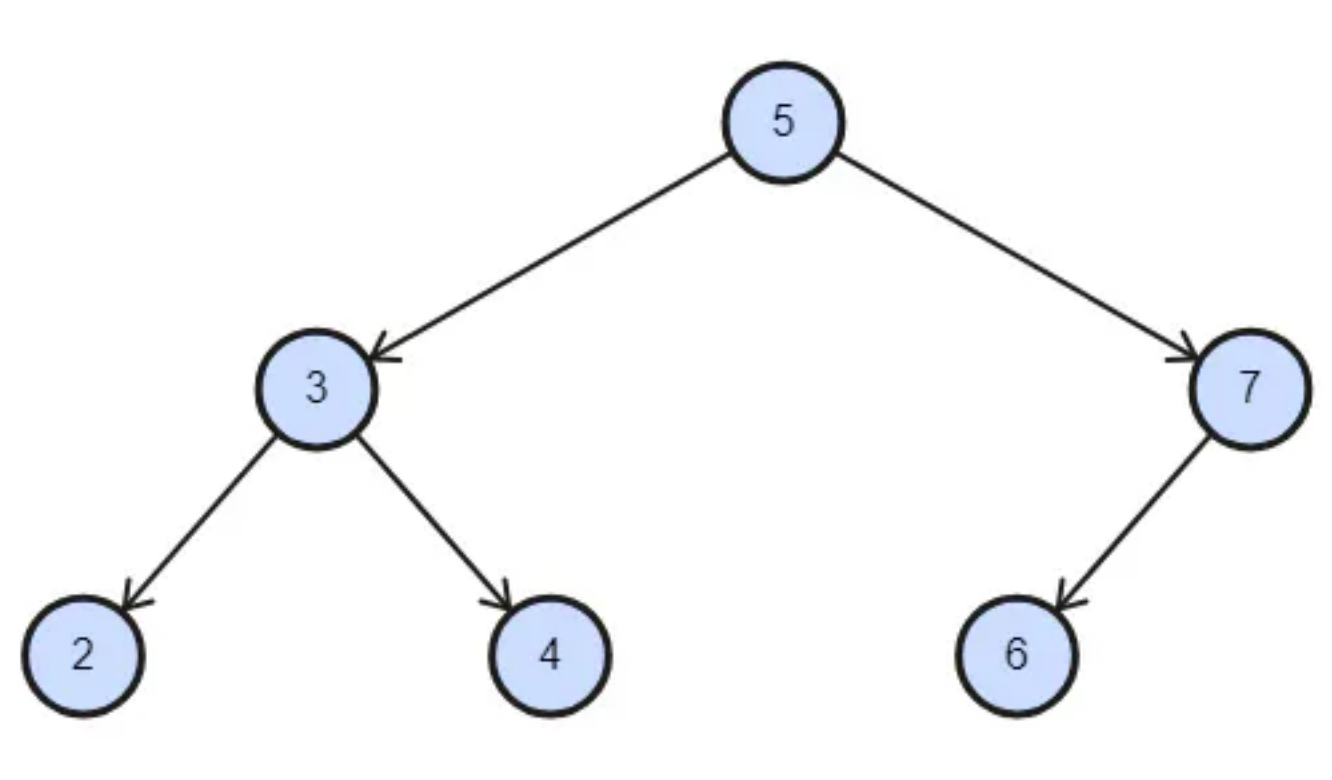
Step 4: Insert 2



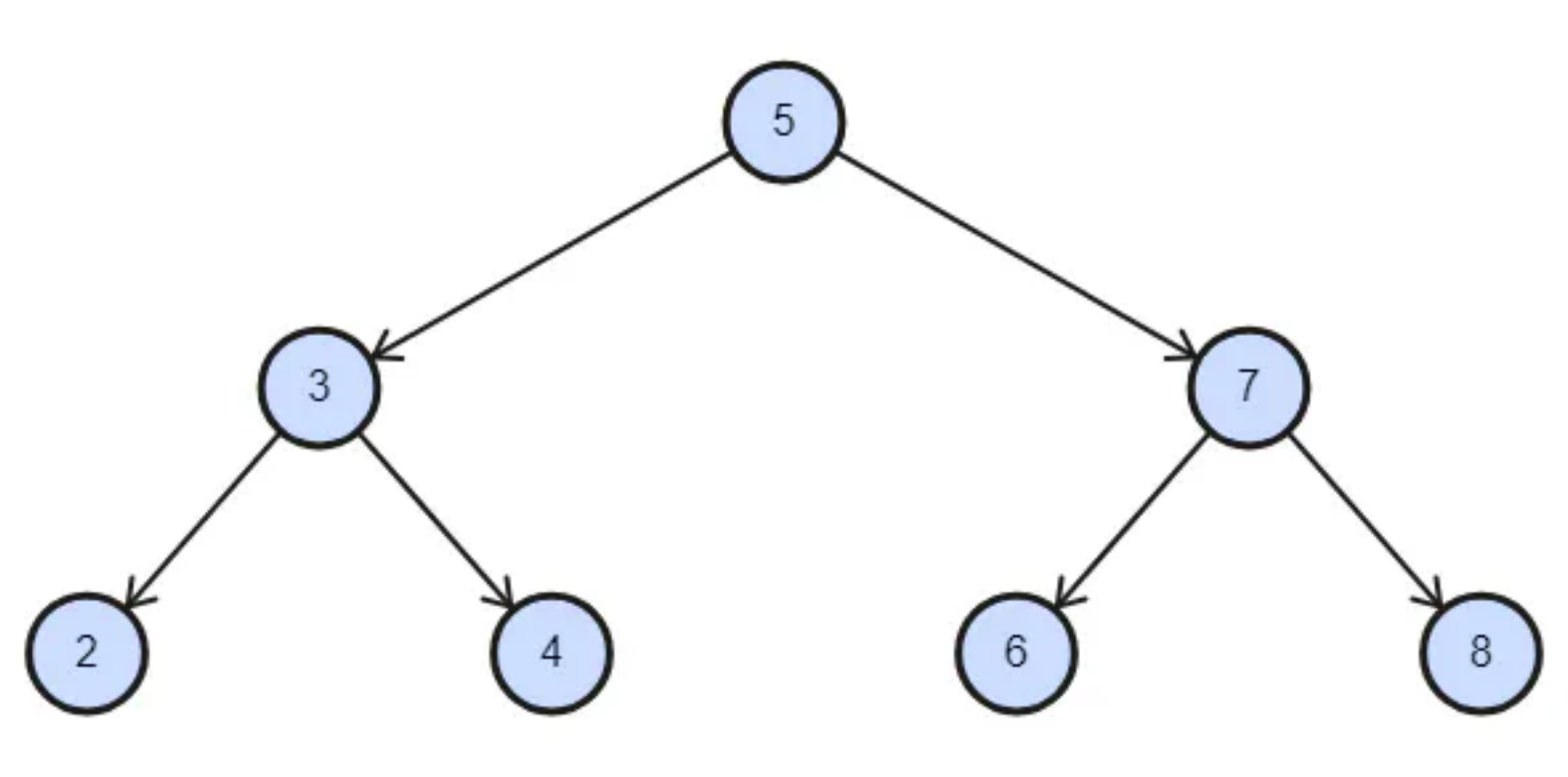
Step 5: Insert 4



Step 6: Insert 6



Step 7: Insert 8



Deletion:

The deletion operation removes a node from the BST while maintaining the BST property. Deletion can be more complex as it involves handling different cases based on the node to be deleted.

The delete method uses the `findMin` and `deleteNode` methods. The code below implements the `findMin`, `deleteNode`, and `deleteMethod` methods.

Java

```java
private Node<T> findMin(Node<T> node) {
    if (node.left == null) {
        return node;
    }
    return findMin(node.left);
}

private Node<T> deleteNode(Node<T> root, T value) {
    if (root == null) {
        return root;
    }

    if (value.compareTo(root.data) < 0) {
        root.left = deleteNode(root.left, value);
    } else if (value.compareTo(root.data) > 0) {
        root.right = deleteNode(root.right, value);
    } else {
        if (root.left == null) {
            return root.right;
        } else if (root.right == null) {
            return root.left;
```

The code provides two functions: `findMin` and `deleteNode,` and a public function `deleteMethod,` that uses the `deleteNode` function to delete a node from the BST.

The `findMin` function is a helper function that finds the minimum node in a given subtree. It starts from the given `node` and traverses the left children until it reaches the leftmost node, which contains the smallest value in the subtree. It returns the pointer to this minimum node.

The `deleteNode` function is another helper function responsible for deleting a node from the BST. It uses recursion to traverse the tree and find the node with the value `node` to be deleted. Once the target node is found, there are three possible cases:

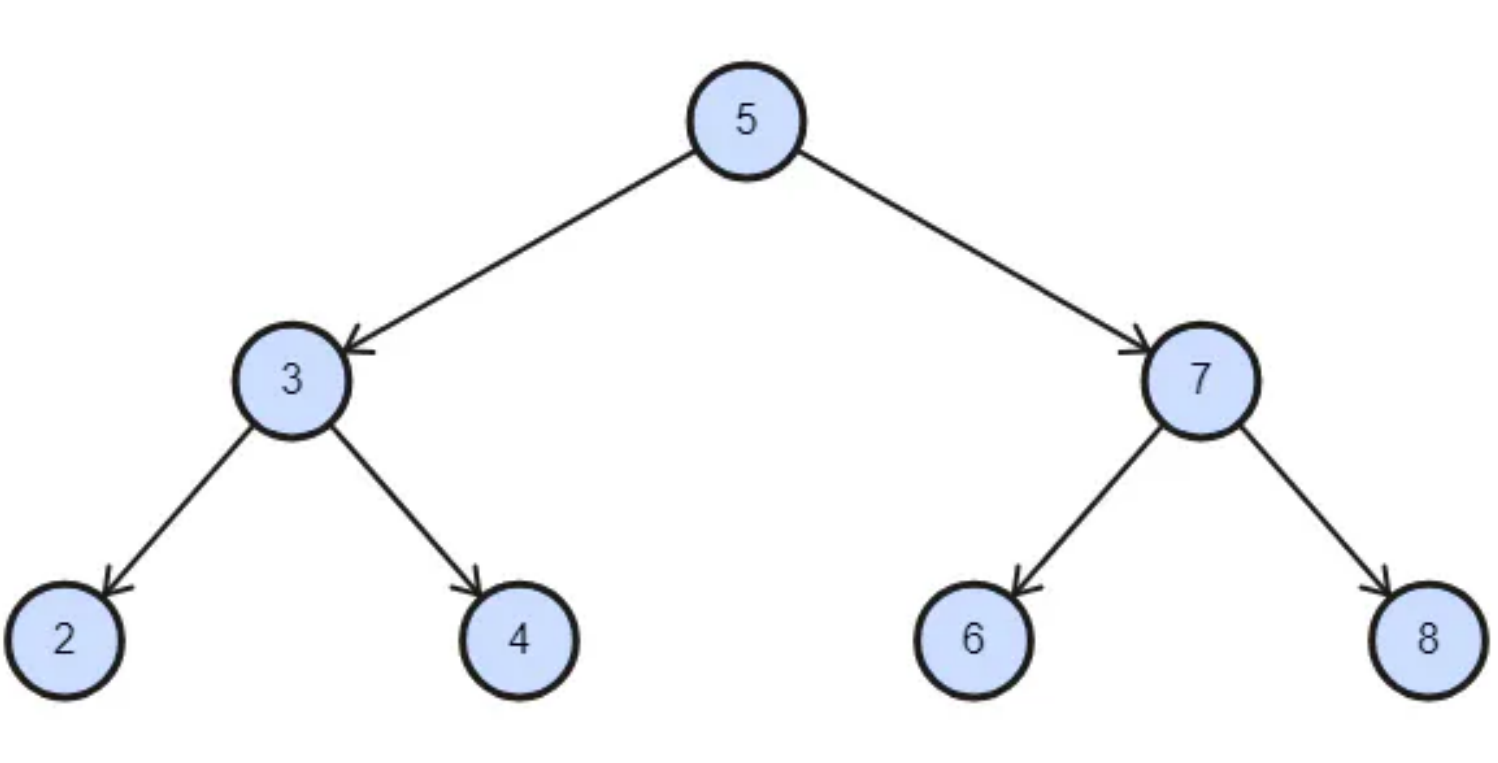
1. The node has no children or only one child (left or right). In this case, the function simply removes the node and links its parent to its child, if any.
2. The node has two children. In this case, the function finds the inorder successor of the node (the minimum node in its right subtree). The value of the inorder successor replaces the value of the node to be deleted, and the function is then recursively called to delete the inorder successor node.

The `deleteMethod` function is the public interface for deleting a node from the BST. It calls the `deleteNode` function, passing the `root` of the BST and the value of the node to be deleted. This initiates the search for the node starting from the root, and the node is deleted if found.

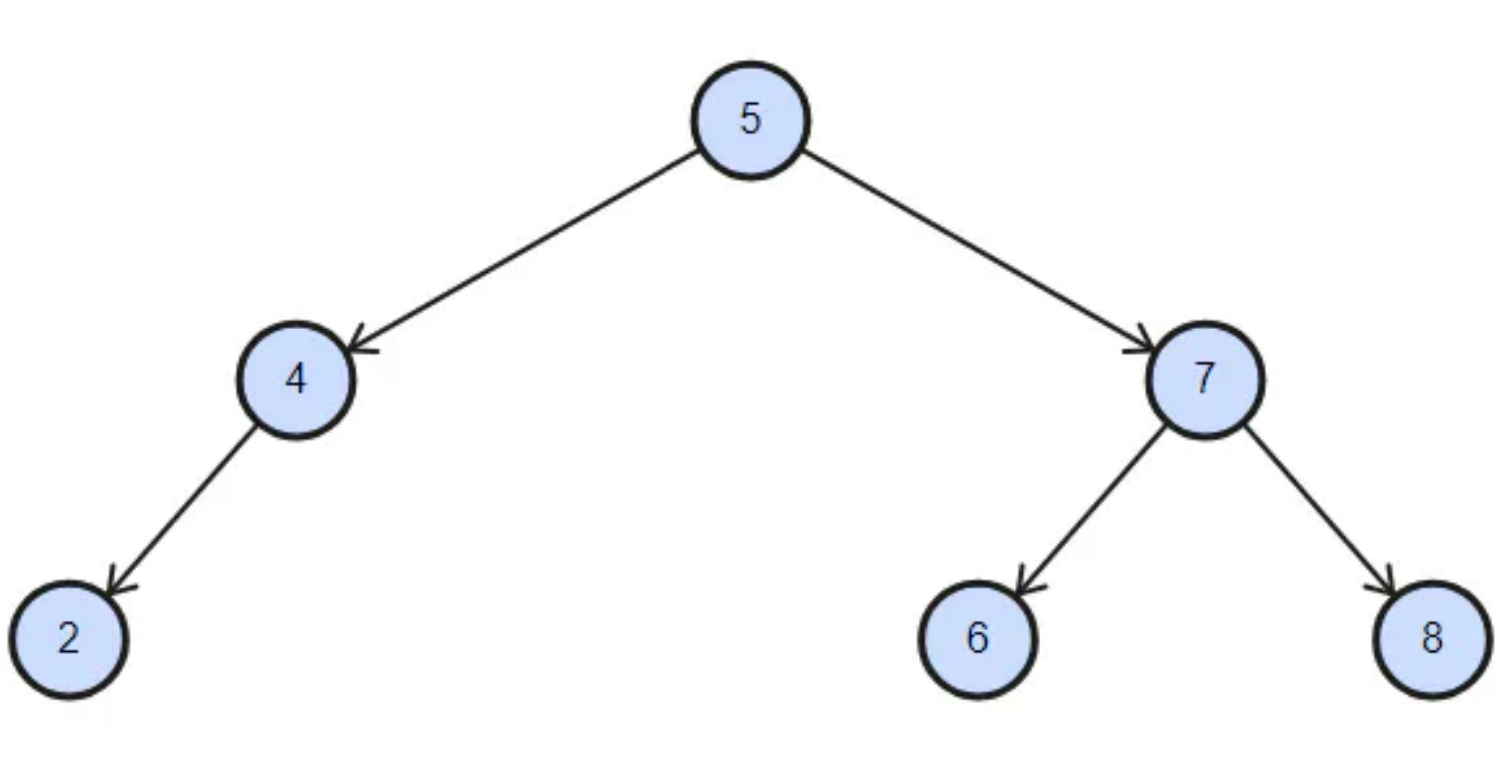
Example:

Let's delete the node with value `3` from the following BST:

Original BST



After deletion of 3



Searching:

Searching for an element in a BST involves traversing the tree based on comparisons between the target value and the node values until the target value is found or the search reaches a leaf node.

Java

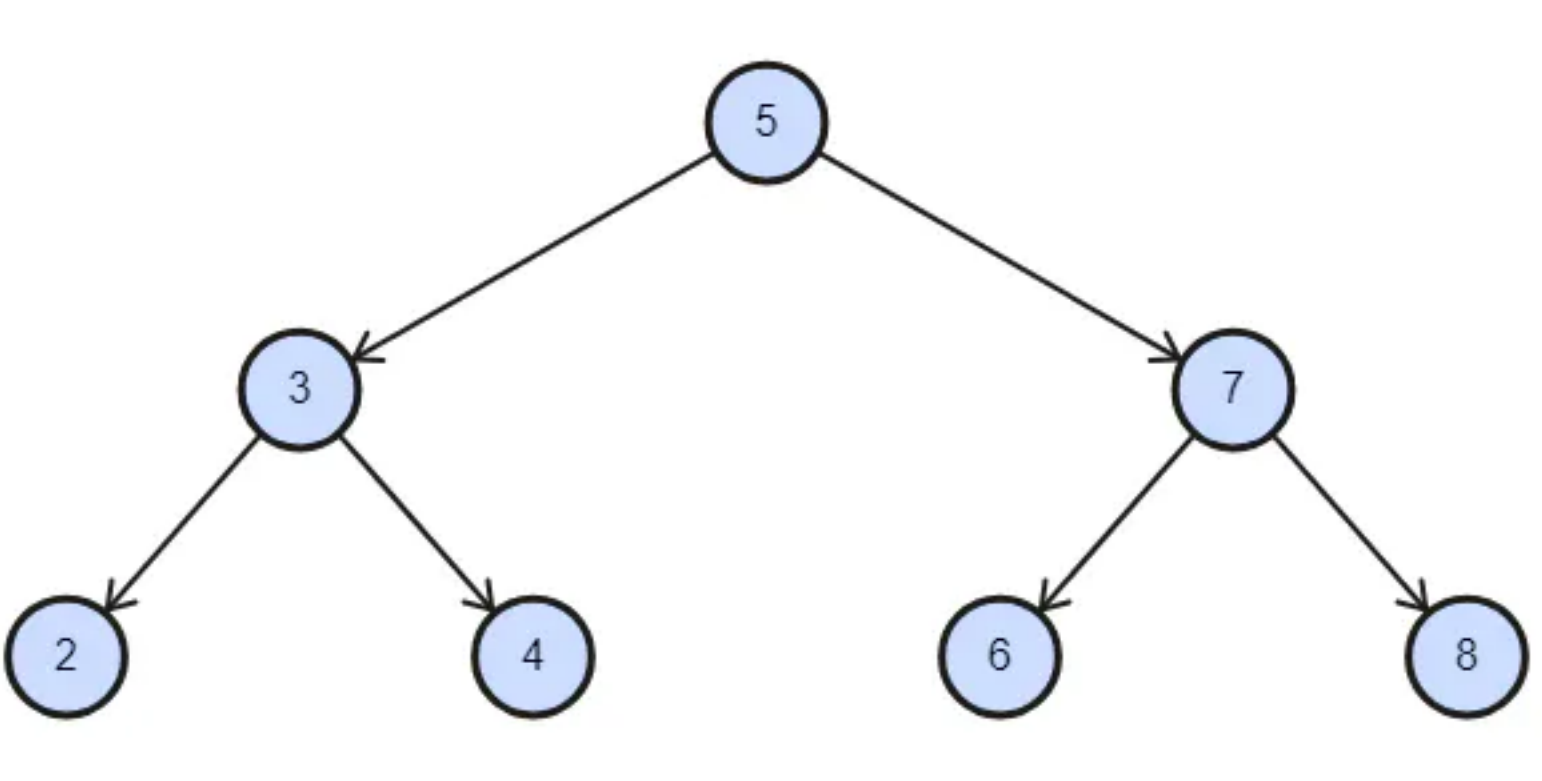
```java
public boolean search(T value) {
    Node<T> current = root;
    while (current != null) {
        if (value.compareTo(current.data) == 0) {
            return true;
        } else if (value.compareTo(current.data) < 0) {
            current = current.left;
        } else {
            current = current.right;
        }
    }
    return false;
}

```

The above code recursively searches for a node with the given `value` in the BST. If the current node is null (empty) or its value matches the `value`, we return the current node. Otherwise, we continue searching in the left or right subtree depending on the comparison between `value` and the current node's value.

Example:

Let's search for the key value `8` in the following BST:



In the above tree, the following steps are performed to search for the desired value:

1. The value `8` is compared with the root value `5`. The traversal is moved to the right subtree since the value is greater than the root value.
2. Again, the value `8` is greater than the right child `7`, and the traversal is moved towards the right side of the tree.
3. The right child contains the desired value.

Traversing a Binary Search Tree (BST)

Traversing a BST involves visiting each node in a specific sequence. There are three common types of BST traversals:

1. In-order Traversal:

In-order traversal visits the left subtree, the current node, and then the right subtree, resulting in the elements being visited in ascending order.

Let's discuss the In-order Traversal C++ implementation :

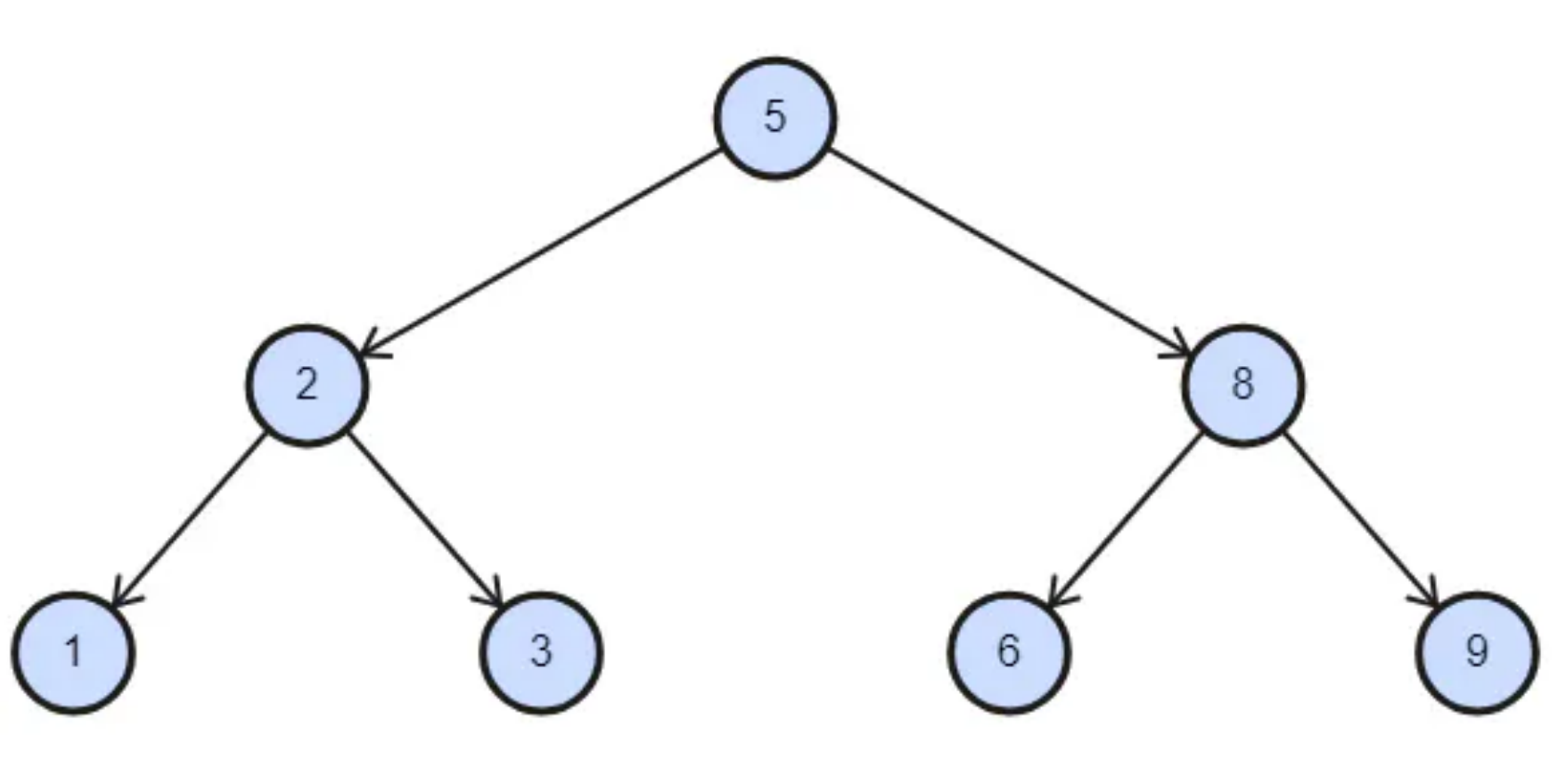
Java

```java
private void inOrderHelper(Node<T> node) {
    if (node != null) {
        inOrderHelper(node.left);
        System.out.print(node.data + " ");
        inOrderHelper(node.right);
    }
}

public void inOrder() {
    inOrderHelper(root);
    System.out.println();
}
```

The above code provides the implementation of in-order traversal for a binary search tree (BST) template class.

1. The function `inOrderHelper()` is a private helper function used for the in-order traversal of the BST. It takes a node `p` to the current node as input and recursively traverses the tree in the following manner:
    - The function proceeds with the traversal if the current node p is not null (i.e., the subtree is not empty).
    - First, it recursively calls `inOrderHelper(p->left)`, which traverses the left subtree of the current node in an in-order fashion.
    - Next, it prints the value of the current node (`p->data`) to the console.
    - Finally, it recursively calls `inOrderHelper(p->right)`, which traverses the right subtree of the current node in an in-order fashion.
2. The `inOrder()` function is a public function that serves as the interface for the in-order traversal of the BST. It starts the in-order traversal from the root of the BST by calling the `inOrderHelper(root)` function.
3. After the `inOrderHelper(root)` call is completed, the `std::cout << std::endl;` statement is used to print a new line, separating the output from any other output in the program.



In-order traversal of this BST results in the sequence: 1, 2, 3, 5, 6, 8, 9.

2. Pre-order Traversal:

Pre-order traversal visits the current node, the left subtree, and the right subtree. Let's discuss the code for the pre-order traversal of a BST.

Java

```java
private void preOrderHelper(Node<T> node) {
    if (node != null) {
        System.out.print(node.data + " ");
        preOrderHelper(node.left);
        preOrderHelper(node.right);
```